



Degree Centrality

✓ [Directed](#) ?✓ [Undirected](#) ?(✓) [Heterogeneous nodes](#) ?(✓) [Heterogeneous relationships](#) ?✓ [Weighted relationships](#) ?

Introduction

The Degree Centrality algorithm can be used to find popular nodes within a graph. The degree centrality measures the number of incoming or outgoing (or both) relationships from a node, which can be defined by the orientation of a relationship projection. If a projection contains directed relationships, only outgoing relationships of a node are counted (the out-degree). For more information on how relationship orientations influence the algorithm result, see the [setting an orientation](#) section. For more general information on relationship orientations, see the [relationship projection syntax](#) section.

It can be applied to either weighted or unweighted graphs. In the weighted case the algorithm computes the sum of all positive weights of adjacent relationships of a node, for each node in the graph. Non-positive weights are ignored.

It can be applied to heterogeneous graphs, however the algorithm will not calculate degree centrality per relationship type. Instead it will treat the graph as homogenous, as indicated by the algorithm traits.

For more information on this algorithm, see:

- [Linton C. Freeman: Centrality in Social Networks Conceptual Clarification, 1979.](#) ↗



AI search

Use-cases

The Degree Centrality algorithm has been shown to be useful in many different applications. For example:

- Degree centrality is an important component of any attempt to determine the most important people in a social network. For example, in BrandWatch's [most influential men and women on Twitter 2017](#)  the top 5 people in each category have over 40m followers each, which is a lot higher than the average degree.
- Weighted degree centrality has been used to help separate fraudsters from legitimate users of an online auction. The weighted centrality for fraudsters is significantly higher because they tend to collude with each other to artificially increase the price of items. Read more in [Two Step graph-based semi-supervised Learning for Online Auction Fraud Detection](#) 

Syntax

This section covers the syntax used to execute the Degree Centrality algorithm in each of its execution modes. We are describing the named graph variant of the syntax. To learn more about general syntax variants, see [Syntax overview](#).

Degree Centrality syntax per mode

Stream mode

Stats mode

Mutate mode

Write mode

Run Degree Centrality in stream mode on a named graph.

CYPHER 

```
CALL gds.degree.stream(  
  graphName: String,  
  configuration: Map  
) YIELD
```

```
nodeId: Integer,  
score: Float
```

Table 1. Parameters

Name	Type	Default	Optional	Description
graphName	String	n/a	no	The name of a graph stored in the catalog.
configuration	Map	{}	yes	Configuration for algorithm-specifics and/or graph filtering.

Table 2. Configuration

Name	Type	Default	Optional	Description
<u>nodeLabels</u>	List of String	['*']	yes	Filter the named graph using the given node labels. Nodes with any of the given labels will be included.
<u>relationshipTypes</u>	List of String	['*']	yes	Filter the named graph using the given relationship types. Relationships with any of the given types will be included.
<u>concurrency</u>	Integer	4 ^[1]	yes	The number of concurrent threads used for running the algorithm.
<u>jobId</u>	String	Generated internally	yes	An ID that can be provided to more easily track the algorithm's progress.
<u>logProgress</u>	Boolean	true	yes	If disabled the progress percentage will not be logged.
orientation	String	NATURAL	yes	The orientation used to compute node degrees. Supported orientations are NATURAL , REVERSE and UNDIRECTED .

1. In a GDS Session, the default is the number of available processors.

Name	Type	Default	Optional	Description
relationshipWeightProperty	String	null	yes	Name of the relationship property to use for weighted degree computation. If unspecified, the algorithm runs unweighted.

1. In a [GDS Session](#), the default is the number of available processors.

Table 3. Results

Name	Type	Description
nodeId	Integer	Node ID.
score	Float	Degree Centrality score.

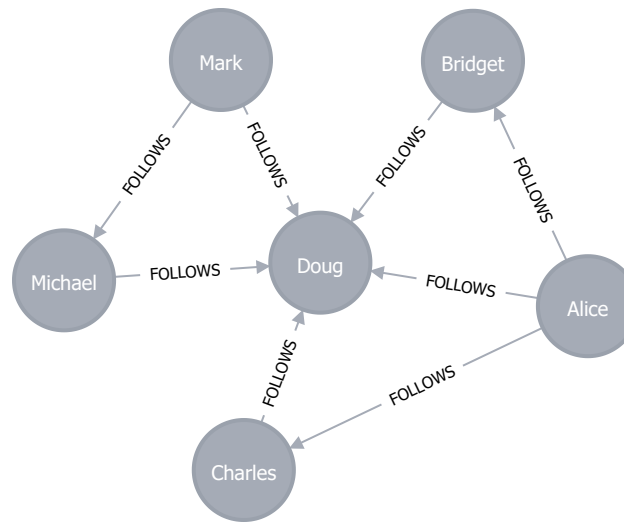
Examples

NOTE

All the examples below should be run in an empty database.

The examples use [Cypher projections](#) as the norm. Native projections will be deprecated in a future release.

In this section we will show examples of running the Degree Centrality algorithm on a concrete graph. The intention is to illustrate what the results look like and to provide a guide in how to make use of the algorithm in a real setting. We will do this on a small social network graph of a handful nodes connected in a particular pattern. The example graph looks like this:



The following Cypher statement will create the example graph in the Neo4j database:

CYPHER

CREATE

```
(alice:User {name: 'Alice'}),
(bridget:User {name: 'Bridget'}),
(charles:User {name: 'Charles'}),
(doug:User {name: 'Doug'}),
(mark:User {name: 'Mark'}),
(michael:User {name: 'Michael'}),

(alice)-[:FOLLOWS {score: 1}]->(doug),
(alice)-[:FOLLOWS {score: -2}]->(bridget),
(alice)-[:FOLLOWS {score: 5}]->(charles),
(mark)-[:FOLLOWS {score: 1.5}]->(doug),
(mark)-[:FOLLOWS {score: 4.5}]->(michael),
(bridget)-[:FOLLOWS {score: 1.5}]->(doug),
(charles)-[:FOLLOWS {score: 2}]->(doug),
(michael)-[:FOLLOWS {score: 1.5}]->(doug)
```

With the graph in Neo4j we can now project it into the graph catalog to prepare it for algorithm execution. We do this using a Cypher projection targeting the `User` nodes and the `FOLLOWS` relationships.

The following statement will project a graph using a reverse projection and store it in the graph catalog under the name 'myGraph'.

CYPHER 

```
MATCH (source:User)-[r:FOLLOWS]->(target:User)
RETURN gds.graph.project(
  'myGraph',
  target,
  source,
  { relationshipProperties: r { .score } }
)
```

For directed relationships, the Degree Centrality algorithm computes the out-degree of each node. In order to retrieve people with the most followers, the graph is projected in a `REVERSE` orientation in the following examples. This will be demonstrated using the Degree Centrality algorithm on this graph.

Memory Estimation

First off, we will estimate the cost of running the algorithm using the `estimate` procedure. This can be done with any execution mode. We will use the `write` mode in this example. Estimating the algorithm is useful to understand the memory impact that running the algorithm on your graph will have. When you later actually run the algorithm in one of the execution modes the system will perform an estimation. If the estimation shows that there is a very high probability of the execution going over its memory limitations, the execution is prohibited. To read more about this, see [Automatic estimation and execution blocking](#).

For more details on `estimate` in general, see [Memory Estimation](#).

The following will estimate the memory requirements for running the algorithm:

CYPHER 

```
CALL gds.degree.write.estimate('myGraph', { writeProperty: 'degree' })
YIELD nodeCount, relationshipCount, bytesMin, bytesMax, requiredMemory
```

Table 13. Results

nodeCount	relationshipCount	bytesMin	bytesMax	requiredMemory
6	8	48	48	"48 Bytes"

Stream

In the `stream` execution mode, the algorithm returns the degree centrality for each node. This allows us to inspect the results directly or post-process them in Cypher without any side effects. For example, we can order the results to find the nodes with the highest degree centrality.

For more details on the `stream` mode in general, see [Stream](#).

The following will run the algorithm in **stream** mode:

CYPHER 

```
CALL gds.degree.stream('myGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score AS followers
ORDER BY followers DESC, name DESC
```

Table 14. Results

name	followers
------	-----------

name	followers
"Doug"	5.0
"Michael"	1.0
"Charles"	1.0
"Bridget"	1.0
"Mark"	0.0
"Alice"	0.0

We can see that Doug is the most popular user in our imaginary social network graph, with 5 followers-all other users follow them, but they don't follow anybody back. In a real social network, celebrities have very high follower counts but tend to follow only very few people. We could therefore consider Doug quite the celebrity!

Stats

In the `stats` execution mode, the algorithm returns a single row containing a summary of the algorithm result. This execution mode does not have any side effects. It can be useful for evaluating algorithm performance by inspecting the `computeMillis` return item. In the examples below we will omit returning the timings. The full signature of the procedure can be found in [the syntax section](#).

For more details on the `stats` mode in general, see [Stats](#).

The following will run the algorithm in **stats** mode:

CYPHER 

```
CALL gds.degree.stats('myGraph')
YIELD centralityDistribution
```



```
RETURN centralityDistribution.min AS minimumScore, centralityDistribution.mean AS meanScore
```

Table 15. Results

minimumScore	meanScore
0.0	1.3333358764648438

Comparing this to the results we saw in the [stream example](#), we can find our minimum and mean values from the table.

Mutate

The `mutate` execution mode extends the `stats` mode with an important side effect: updating the named graph with a new node property containing the degree centrality for that node. The name of the new property is specified using the mandatory configuration parameter `mutateProperty`. The result is a single summary row, similar to `stats`, but with some additional metrics. The `mutate` mode is especially useful when multiple algorithms are used in conjunction.

For more details on the `mutate` mode in general, see [Mutate](#).

The following will run the algorithm in **mutate** mode:

CYPHER 

```
CALL gds.degree.mutate('myGraph', { mutateProperty: 'degree' })
YIELD centralityDistribution, nodePropertiesWritten
RETURN centralityDistribution.min AS minimumScore, centralityDistribution.mean AS meanScore, nodePropertiesWritten
```

Table 16. Results

minimumScore	meanScore	nodePropertiesWritten
0.0	1.3333358764648438	6

The returned result is the same as in the `stats` example. Additionally, the graph 'myGraph' now has a node property `degree` which stores the degree centrality score for each node. To find out how to inspect the new schema of the in-memory graph, see [Listing graphs in the catalog](#).

Write

The `write` execution mode extends the `stats` mode with an important side effect: writing the degree centrality for each node as a property to the Neo4j database. The name of the new property is specified using the mandatory configuration parameter `writeProperty`. The result is a single summary row, similar to `stats`, but with some additional metrics. The `write` mode enables directly persisting the results to the database.

For more details on the `write` mode in general, see [Write](#).

The following will run the algorithm in **write** mode:

CYPHER 

```
CALL gds.degree.write('myGraph', { writeProperty: 'degree' })
YIELD centralityDistribution, nodePropertiesWritten
RETURN centralityDistribution.min AS minimumScore, centralityDistribution.mean AS meanScore, nodePropertiesWritten
```

Table 17. Results

minimumScore	meanScore	nodePropertiesWritten
--------------	-----------	-----------------------

minimumScore	meanScore	nodePropertiesWritten
0.0	1.3333358764648438	6

The returned result is the same as in the `stats` example. Additionally, each of the seven nodes now has a new property `degree` in the Neo4j database, containing the degree centrality score for that node.

Weighted Degree Centrality example

This example will explain the weighted Degree Centrality algorithm. This algorithm is a variant of the Degree Centrality algorithm, that measures the sum of positive weights of incoming and outgoing relationships.

The following will run the algorithm in **stream** mode, showing which users have the highest weighted degree centrality:

CYPHER 

```
CALL gds.degree.stream(
  'myGraph',
  { relationshipWeightProperty: 'score' }
)
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score AS weightedFollowers
ORDER BY weightedFollowers DESC, name DESC
```

Table 18. Results

name	weightedFollowers
"Doug"	7.5
"Charles"	5.0

name	weightedFollowers
"Michael"	4.5
"Mark"	0.0
"Bridget"	0.0
"Alice"	0.0

Doug still remains our most popular user, but there isn't such a big gap to the next person. Charles and Michael both only have one follower, but those relationships have a high relationship weight. Note that Bridget also has a weighted score of 0.0, despite having a connection from Alice. That is because the `score` property value between Bridget and Alice is negative and will be ignored by the algorithm.

Setting an orientation

By default, node centrality uses the `NATURAL` orientation to compute degrees. For some use-cases it makes sense to analyze a different orientation, for example, if we want to find out how many users follow another user. In order to change the orientation, we can use the `orientation` configuration key. There are three supported values:

- `NATURAL` (default) corresponds to computing the out-degree of each node.
- `REVERSE` corresponds to computing the in-degree of each node.
- `UNDIRECTED` computes and sums both the out-degree and in-degree of each node.

The following will run the algorithm in `stream` mode, showing which users have the highest in-degree centrality using the reverse orientation of the relationships:

CYPHER 

```
CALL gds.degree.stream(  
  'myGraph',  
  { orientation: 'REVERSE' }  
)  
YIELD nodeId, score  
RETURN gds.util.asNode(nodeId).name AS name, score AS followees  
ORDER BY followees DESC, name DESC
```

Table 19. Results

name	followees
"Alice"	3.0
"Mark"	2.0
"Michael"	1.0
"Charles"	1.0
"Bridget"	1.0
"Doug"	0.0

The example shows that when looking at the reverse orientation, Alice is more central in the network than Doug .

Contents

- Introduction
- Use-cases
- Syntax
- Examples
 - Memory Estimation
 - Stream
 - Stats
 - Mutate
 - Write
- Weighted Degree Centrality example
- Setting an orientation

Graph Data Science courses on GraphAcademy

Follow our Graph Data Analytics learning path to learn how to apply graph thinking to your machine learning pipelines.

[Enroll now](#)

LEARN

-  [Sandbox](#)
-  [Neo4j Community Site](#)
-  [Neo4j Developer Blog](#)
-  [Neo4j Videos](#)
-  [GraphAcademy](#)
-  [Neo4j Labs](#)

SOCIAL

-  [Twitter](#)
-  [Meetups](#)
-  [Github](#)
-  [Stack Overflow](#)

[Want to Speak?](#)

CONTACT US →

US: 1-855-636-4532
Sweden +46 171 480 113
UK: +44 20 3868 3223
France: +33 (0) 1 88 46 13 20

© 2026 Neo4j, Inc.

[Terms](#) | [Privacy](#) | [Sitemap](#)

Neo4j[®], Neo Technology[®], Cypher[®], Neo4j[®] Bloom™ and Neo4j[®] Aura™ are registered trademarks of Neo4j, Inc. All other marks are owned by their

respective companies.

